

Vietnam National University – Ho Chi Minh City
International University

Advanced Database System (IT502IU)

Final Report

HCMIU Map with Multiple Contributors and Live Updates

Project Topic ID & Name: Individual DBMS Application Project

Student: Huynh Tran Khanh
Student ID: MITIU25210

Instructor: Nguyen Thi Thuy Loan

March 11, 2026

Contents

Abstract	iv
1 Introduction	v
1.1 System overview	v
1.2 Project objectives	v
1.3 Problem context and motivation	v
1.4 Related work and literature context	vi
1.4.1 Indoor navigation systems	vi
1.4.2 Graph databases and multi-model systems	vi
1.4.3 Consistency models and real-time collaboration	vi
1.4.4 Pathfinding algorithms	vii
1.5 Techniques and tools used	vii
1.6 Scope and limitations	vii
1.7 Report structure	vii
2 Project Timeline & Contribution	viii
2.1 Timeline of milestones	viii
2.2 Work breakdown by phase	viii
2.3 Contribution statement	ix
3 Methodology	x
3.1 Selected DBMS and justification	x
3.1.1 Comparison with alternative approaches	x
3.2 Research-grounded design considerations	x
3.2.1 Connected-data modeling	x
3.2.2 Consistency and real-time behavior	xi
3.2.3 Algorithmic foundations for navigation	xi
3.3 Database design	xi
3.3.1 Collection architecture	xi
3.3.2 Conceptual ER-style view	xii
3.3.3 Relational projection (3NF analysis)	xii
3.4 Authentication architecture	xiii
3.4.1 Threat model and design goals	xiii
3.4.2 Protocol description	xiii
3.4.3 Security properties	xiii
3.5 Real-time event system	xiv
3.5.1 WebSocket architecture	xiv
3.5.2 Event types and broadcast semantics	xiv

3.6	Data collection and population	xiv
3.7	Indexing and query-planning strategy	xv
3.8	Database queries and operations	xv
3.8.1	Relational algebra formalization	xv
3.8.2	AQL query implementations	xv
3.9	Trial system design	xvii
3.9.1	Motivation and governance model	xvii
3.9.2	Trial state machine	xvii
3.9.3	Data integrity constraints	xvii
3.10	Application architecture	xviii
3.10.1	System components	xviii
3.10.2	API endpoint summary	xviii
3.10.3	Map data structure and rendering	xix
3.11	User interface design	xix
4	Discussion & Implementation	xxii
4.1	Implementation process	xxii
4.2	Testing methodology	xxii
4.2.1	API test suite	xxii
4.2.2	End-to-end test suite	xxiii
4.2.3	Demo video scenarios	xxiii
4.3	Results and analysis	xxiii
4.3.1	Functional outcomes	xxiii
4.3.2	Database-level observations	xxiv
4.3.3	Algorithmic component analysis	xxiv
4.4	Challenges and solutions	xxiv
4.5	Threats to validity	xxv
5	Conclusion & Future Work	xxvi
5.1	Summary of key findings	xxvi
5.2	Reflection on objectives	xxvi
5.3	Advanced database implications	xxvii
5.4	Future work	xxvii

Abstract

This report presents the design, implementation, and evaluation of an ArangoDB-backed collaborative campus map system developed for Ho Chi Minh City International University (HCMIU). The project integrates two major capabilities: (1) algorithmic indoor navigation—encompassing single-source shortest-path computation via breadth-first search and multi-stop route optimization via bitmask dynamic programming (the Held–Karp algorithm)—and (2) a graph-oriented social knowledge layer in which rooms, users, posts, comments, and trial artifacts are linked as first-class entities connected by typed edges in a property-graph model.

The implementation combines a Vite/TypeScript single-page application frontend with a Node.js/Express 5 backend, using ArangoDB 3.11 as the core multi-model database and WebSockets (RFC 6455) for real-time event propagation. From a database perspective, the most significant contribution is an explicit document-and-edge design: content nodes reside in five document collections (`users`, `entities`, `follows`, `notifications`, `trials`) while semantic links are persisted in a dedicated edge collection (`entity_references`), enabling relationship-centric queries such as reference discovery, full-text exploration, shortest connection paths between arbitrary entities, and notification fan-out to followers.

Beyond implementation details, this report provides a research-grounded analysis comparing multi-model database design to polyglot persistence trade-offs, discusses consistency and real-time delivery semantics in the context of eventual consistency theory, examines graph query workloads in terms of computational complexity and developer maintainability, and evaluates the security properties of the SRP-inspired authentication protocol employed. The report documents the complete system architecture, schema decisions, indexing and data-generation strategy, representative algebraic and AQL query operations, implementation outcomes, testing methodology, and limitations. The system demonstrates that a multi-model DBMS can substantially reduce impedance mismatch in connected applications compared with separating document storage and graph traversal into different engines, while still requiring careful operational design around access control, indexing strategy, event ordering, and conflict resolution.

Chapter 1

Introduction

1.1 System overview

The system is a web-based collaborative map platform designed for university indoor spaces. Users can browse seven campus floors, compute shortest paths between any two points, plan optimized multi-stop routes using a traveling-salesman solver, and open location-linked discussion threads. Beyond mapping, the platform provides a full suite of social and governance workflows: entity creation with cross-references, follow/notification mechanics, an activity feed, and a structured “court trial” mechanism for community-driven dispute resolution around shared campus knowledge.

The platform is deployed as a Docker Compose stack comprising three services: an ArangoDB 3.11 database, a Node.js backend serving both the REST API and WebSocket connections, and a Vite-built static frontend. This architecture enables single-command deployment and ensures environmental consistency across development, testing, and production-like scenarios.

1.2 Project objectives

The project objectives are:

1. Build an interactive map experience suitable for wayfinding in complex indoor environments, consistent with wayfinding research emphasizing reduced cognitive load and spatial orientation support in large facilities [12].
2. Demonstrate advanced database features using ArangoDB’s native multi-model architecture (document storage, graph traversal, and full-text-like querying within a single query language) [3].
3. Provide relationship-first collaborative features—including entity references, degree-of-separation analysis, full-text search, and event-driven notifications—that exploit the graph model natively.
4. Implement a secure, password-safe authentication system using client-side key derivation with memory-hard functions to protect user credentials.
5. Validate system correctness through comprehensive automated API test suites and end-to-end browser scenarios that exercise all major workflows.

1.3 Problem context and motivation

Campus-scale indoor information systems exhibit two data characteristics that are often in tension: *high schema variability* (new kinds of room notes, comments, social interactions, moderation records emerge as features evolve) and *high relationship density* (references between entities, follows, replies, dispute participants, notification audiences, and map-location linkages form a densely connected graph). Traditional architectural approaches that split these concerns across

separate specialized systems—for example, a document store for content and a separate graph database for relationships—can increase operational overhead, query complexity, and data synchronization burden. This challenge aligns with the broader argument that one-size-fits-all data management is rarely optimal for modern applications [10].

The *polyglot persistence* pattern, while architecturally sound in theory, introduces concrete operational costs: separate deployment pipelines, independent backup and recovery procedures, cross-system transaction coordination, and duplicated client libraries [9]. For a campus-oriented application with limited operational resources, these costs can outweigh the theoretical benefits of workload-specific optimization.

For this project, the primary motivation was to evaluate whether a single multi-model engine—ArangoDB, which natively supports document, graph, and key-value workloads in one process—could support both rapid feature evolution and relationship-heavy analytics without introducing external middleware. A secondary motivation was to study how graph-native query patterns (traversals, shortest paths, edge aggregations) compare in expressiveness and maintainability to equivalent operations in a relational or document-only system.

The platform therefore serves a dual purpose: it is both a *practical campus tool* (wayfinding, knowledge sharing, governance) and an *applied DBMS experiment* focused on model fit, developer productivity, query expressiveness, and architectural simplicity.

1.4 Related work and literature context

1.4.1 Indoor navigation systems

Indoor navigation research has explored diverse approaches from sensor-based positioning (Wi-Fi fingerprinting, BLE beacons, visual SLAM) to graph-based pathfinding over pre-surveyed floor plans [12]. Our approach belongs to the latter category: the campus is modeled as a set of floor-specific graphs where nodes represent points of interest (rooms, corridors, stairs, lifts) and edges represent walkable connections. This graph-based approach trades real-time positioning accuracy for deployment simplicity and deterministic routing correctness.

1.4.2 Graph databases and multi-model systems

The property-graph model, formalized by Angles and Gutiérrez [1], represents data as nodes with properties connected by typed, directed edges that also carry properties. This model naturally captures the semantic relationships in collaborative systems. Robinson et al. [8] argue that graph databases offer significant advantages for connected data, particularly when relationship queries (joins in relational terms) dominate the workload.

ArangoDB positions itself as a multi-model database that unifies document, graph, and key-value access patterns under a single query language (AQL) and storage engine [3, 2]. This contrasts with the polyglot persistence approach advocated by Sadalage and Fowler [9], where different data stores are chosen for different workload characteristics. Our project empirically tests the multi-model hypothesis in a real application context.

1.4.3 Consistency models and real-time collaboration

Vogels [11] provides a foundational taxonomy of consistency models in distributed systems, ranging from strong consistency to eventual consistency. Our system employs a pragmatic hybrid: REST API writes provide strong consistency (the server confirms persistence before responding), while WebSocket-driven UI updates follow an eventual-consistency model where clients may observe stale state for a short window before receiving broadcast notifications. This design mirrors patterns seen in modern collaborative applications such as Google Docs and Figma, where optimistic UI updates are reconciled with authoritative server state.

1.4.4 Pathfinding algorithms

The shortest-path component relies on breadth-first search (BFS), which is optimal for unweighted graphs. For weighted scenarios, Dijkstra’s algorithm [5] provides $O((V + E) \log V)$ optimal solutions. The multi-stop route planner uses the Held–Karp algorithm [4, 7], a bitmask dynamic-programming formulation of the Traveling Salesman Problem (TSP) with $O(2^n \cdot n^2)$ time complexity, which is tractable for the campus-scale instances encountered in practice ($n \leq 15$).

1.5 Techniques and tools used

The implementation stack includes:

- **DBMS:** ArangoDB 3.11 (multi-model: document + graph + key-value).
- **Backend:** Node.js 20+ with Express 5, arangojs 10 driver, and `ws` WebSocket library.
- **Authentication:** `libsodium-wrappers` (Argon2id key derivation) with server-side SHA-256 verification.
- **Realtime:** WebSocket protocol per RFC 6455 [6] with per-user socket tracking.
- **Frontend:** Vite 5 build system, TypeScript, Hyperscript for DOM generation, Tailwind CSS for styling, `Anime.js` for path animations.
- **Pathfinding:** BFS for single-floor shortest path, bitmask DP (Held–Karp) for TSP.
- **Testing:** Node.js native test runner for API tests, Puppeteer 24 for end-to-end browser scenarios and screencast-based demo video generation.
- **Containerization:** Docker and Docker Compose for reproducible multi-service deployment.

1.6 Scope and limitations

Scope. The project covers indoor map visualization across seven floors, single-source and multi-stop route planning, collaborative entity creation with cross-references, follow/notification mechanics, a structured trial/dispute-resolution workflow, graph-oriented research APIs (reference lookup, full-text search, degree of separation), an activity feed, and comprehensive automated testing.

Limitations.

- No GPS, Wi-Fi fingerprinting, or RTLS integration for live indoor positioning—routing relies on pre-defined graph topology.
- Full-text search uses case-insensitive substring matching via AQL `CONTAINS` rather than a dedicated ranked information retrieval engine (e.g., ArangoSearch with BM25).
- Routing accuracy depends on graph modeling fidelity; missing connectors, inaccurate floor transitions, or stale room metadata degrade path quality.
- No formal throughput or latency benchmarks were conducted; performance observations are qualitative.
- The trial system is a governance experiment and is not intended as a legal or disciplinary framework.

1.7 Report structure

Chapter 2 presents the project timeline and personal contribution. Chapter 3 details the methodology, including DBMS rationale, data design, authentication architecture, real-time event system, algorithmic implementations, and representative database operations. Chapter 4 discusses implementation outcomes, testing methodology, challenges encountered, and threats to validity. Chapter 5 concludes and proposes future work directions.

Chapter 2

Project Timeline & Contribution

2.1 Timeline of milestones

Week	Phase	Outcome
Week 1	Design & orchestration	Defined system architecture, Docker Compose orchestration (ArangoDB + backend + frontend), initial collection schema, and entity model design.
Week 2	Backend API implementation	Implemented SRP-inspired authentication, entity CRUD with reference edge synchronization, follow/unfollow mechanics, notification generation, trial lifecycle APIs, and research endpoints.
Week 3	Frontend integration	Connected collaborative UI pages (Hub, Auth, Entities, Trials, Research, Notifications, Activity), implemented map-to-entity deep linking, WebSocket real-time refresh, and pathfinding algorithms (BFS + Held–Karp TSP).
Week 4	Testing, demos & reporting	Developed comprehensive API test suite (243 lines), E2E test suite (295 lines), five Puppeteer-based demo video scenarios (2,100+ lines), and completed final documentation.

2.2 Work breakdown by phase

- **Design:** Determined entity model with typed documents, edge semantics for references, event model for WebSocket broadcasts, and authentication protocol with client-side key derivation.
- **Implementation:** Developed 28 REST API endpoints, WebSocket server with per-user socket tracking, seven floor map layouts with graph connectivity, and pathfinding algorithms.
- **Experimentation:** Executed scenario tests covering authentication flows, entity lifecycle, follow/notification behavior, trial state machine transitions, research queries, and map navigation.
- **Documentation:** Produced comprehensive API documentation (`docs/collaborative-api.md`), user tutorial (`docs/tutorial.md`), system manifesto (`MANIFESTO.md`), and this final report.

2.3 Contribution statement

All project stages—design, coding, testing, deployment configuration, documentation, and report writing—were completed individually by the student.

Chapter 3

Methodology

3.1 Selected DBMS and justification

ArangoDB was selected because the application requires both flexible document storage (users, entities, trials, notifications with evolving schemas) and efficient graph traversal over explicit references and follows. A dedicated property-graph database (e.g., Neo4j) would require a separate system for document-centric operations and schema flexibility, while a document-only database (e.g., MongoDB) would necessitate application-level graph traversal logic or expensive recursive lookups for path queries. In contrast, ArangoDB natively supports document and graph workloads with AQL in a single engine [2, 3], aligning with graph database principles for highly connected data [1, 8].

A second rationale is architectural simplicity under continuous iteration. During development, feature additions—trial negotiation history with proposal tracking, map-linked collaborative comments, follower-based notification fan-out—altered both entity structure and relationship depth simultaneously. A multi-model database reduced migration friction by allowing document enrichment (adding new fields, nested objects) without sacrificing graph traversal capabilities. This design choice follows broader system research that warns against forcing all workloads into a single rigid model while still emphasizing careful workload-oriented tuning [10].

3.1.1 Comparison with alternative approaches

Table 3.1 summarizes the trade-offs between the multi-model approach and two common alternatives.

Table 3.1: DBMS approach comparison for the campus map application.

Criterion	Multi-model (ArangoDB)	Polyglot (MongoDB + Neo4j)	Relational (PostgreSQL)
Schema flexibility	Native JSON documents	MongoDB documents	Requires ALTER TABLE
Graph traversal	Native AQL traversal	Native Cypher	Recursive CTEs
Operational overhead	Single deployment	Two separate systems	Single deployment
Transaction scope	Single-engine ACID	Cross-system coordination	Full ACID
Query language	AQL (unified)	Two languages	SQL only
Learning curve	Moderate	High (two systems)	Low–moderate

3.2 Research-grounded design considerations

3.2.1 Connected-data modeling

The platform adopts a property-graph mindset for interaction semantics: references and follows are first-class edges stored in dedicated collections rather than embedded arrays within documents. This design enables path-based reasoning (e.g.,

degree-of-separation queries, reference chain discovery) with explicit traversal semantics in AQL, which would be difficult to maintain in purely denormalized document structures as relationship depth and query complexity grow [8].

The choice to model references as edges rather than embedded arrays has concrete implications: (1) bidirectional traversal is supported without maintaining inverse arrays, (2) edge-level metadata (creation timestamp, type) can be attached independently, and (3) graph algorithms (shortest path, connected components) can operate directly on the edge collection without application-level transformation.

3.2.2 Consistency and real-time behavior

The collaboration experience uses a pragmatic hybrid consistency model. REST API endpoints persist canonical state with strong consistency guarantees: the server confirms successful database writes before returning HTTP responses to clients. WebSocket events then notify connected clients to refresh or patch their local views. This approach prioritizes responsiveness while accepting short-lived read-after-write lag on clients that have not yet received the broadcast event—a trade-off consistent with eventual-consistency discussions in distributed systems [11].

In practice, three mechanisms reduce ambiguity when multiple users interact concurrently:

1. **Deterministic IDs:** Map entities use SHA-256-based deterministic identifiers derived from floor number and room name, preventing duplicate creation and enabling stable cross-references.
2. **Server-side timestamps:** All `createdAt` fields are set by the server, ensuring consistent temporal ordering regardless of client clock drift.
3. **Full-state refresh:** On receiving a WebSocket event, clients re-fetch the complete entity/trial/notification state rather than applying incremental patches, eliminating state divergence at the cost of slightly higher bandwidth.

3.2.3 Algorithmic foundations for navigation

Indoor routing components rely on graph search algorithms with well-understood complexity guarantees:

Single-floor shortest path. The system uses breadth-first search (BFS) on the unweighted floor graph. BFS guarantees optimal shortest paths in $O(V + E)$ time for unweighted graphs, where V is the number of navigable points and E is the number of walkable connections on a floor. For the HCMIU campus, typical floor graphs contain 40–60 nodes and 50–80 edges, making BFS effectively instantaneous.

Multi-floor routing. When source and destination are on different floors, the system evaluates all available lift positions as intermediate transfer points. For each lift, it computes the shortest path from the source to the lift on the source floor and from the lift to the destination on the target floor, selecting the combination with minimum total path length. With four lifts in the campus, this requires at most eight BFS executions per cross-floor query.

Multi-stop route optimization. The traveling-salesman planner uses the Held–Karp algorithm [4, 7], a bitmask dynamic-programming formulation that computes the optimal visitation order for n destinations in $O(2^n \cdot n^2)$ time and $O(2^n \cdot n)$ space. This is tractable for campus-scale instances: for $n = 15$ stops, the DP table contains $15 \times 2^{15} = 491,520$ entries, which is expected to complete well within interactive response times on modern hardware. The algorithm guarantees an optimal solution, unlike greedy or heuristic TSP approximations.

3.3 Database design

3.3.1 Collection architecture

The database uses six collections: five document collections and one edge collection. Table 3.2 summarizes the schema.

Table 3.2: ArangoDB collection schema.

Collection	Type	Key Attributes
users	Document	id, username (unique), clientSalt, serverSalt, passwordHash, createdAt
entities	Document	id, type (post/comment/map_location/trial), title, body, parentEntityId, references[], tags[], createdBy, createdAt
entity_references	Edge	_from (entities/X), _to (entities/Y), _key (SHA-256 hash), createdAt
follows	Document	userId, entityId, createdAt
notifications	Document	id, userId, entityId, type, read (boolean), createdAt
trials	Document	id, entityId, plaintiffUserId, defendantUserId, status, agreedJudges[], votes{}, outcome, judgeNegotiationHistory[], lastProposedBy, lastProposedJudges[], createdAt

3.3.2 Conceptual ER-style view

The conceptual data model defines four primary entity types and two relationship types:

- **User:** Account identity with cryptographic authentication metadata (salts and derived password hash).
- **Entity:** A polymorphic content node that serves as the universal unit of discourse. Entity types include `post` (top-level discussion), `comment` (reply attached to a parent entity), `map_location` (auto-generated room/facility thread), and `trial` (dispute-linked discussion entity).
- **Trial:** A structured dispute case with plaintiff, defendant, negotiated judge panel, votes, and resolution outcome. Each trial is linked to a companion entity for threaded discussion.
- **Notification:** An event record delivered to a user when activity occurs on entities they follow.

Relations:

- **entity_references** (edge collection): Directed semantic links between entities. When a user creates or updates an entity, the `references[]` array is synchronized to the edge collection, enabling bidirectional graph traversal. Edge keys are deterministic SHA-256 hashes of the source–target pair, ensuring idempotent upserts.
- **follows:** A user follows an entity to receive notifications on future activity. Stored as a document with composite uniqueness on (`userId`, `entityId`).

3.3.3 Relational projection (3NF analysis)

Although deployed in ArangoDB’s document model, an equivalent normalized relational schema illustrates the underlying data dependencies:

- Users(user_id, username [UNIQUE], client_salt, server_salt, password_hash, created_at)
- Entities(entity_id, type, title, body, parent_entity_id [FK→Entities], created_by [FK→Users], created_at)
- EntityReferences(from_entity_id [FK], to_entity_id [FK], created_at)
- Follows(user_id [FK], entity_id [FK], created_at)
- Notifications(notification_id, user_id [FK], entity_id [FK], type, read, created_at)
- Trials(trial_id, entity_id [FK], plaintiff_id [FK], defendant_id [FK], status, outcome, created_at)

This projection is in Third Normal Form (3NF): every non-key attribute depends on the whole key and nothing but the key. The `agreedJudges`, `votes`, and `judgeNegotiationHistory` fields, which are JSON arrays/objects in the document model, would require additional junction tables (`TrialJudges`, `TrialVotes`, `NegotiationHistory`) in a fully normalized relational schema.

3.4 Authentication architecture

3.4.1 Threat model and design goals

The authentication system is designed with two primary goals: (1) the server never receives or stores the user's plaintext password, and (2) stored credentials are resistant to offline brute-force attacks even if the database is compromised.

3.4.2 Protocol description

The system implements a two-phase, SRP-inspired (Secure Remote Password) authentication protocol using libsodium's Argon2id key derivation:

Signup flow:

1. Client sends username to `POST /api/auth/signup/start`.
2. Server generates two cryptographically random 16-byte salts (`clientSalt`, `serverSalt`) and returns them.
3. Client derives a 32-byte key from the password using Argon2id with `clientSalt`:

```
clientHash = argon2id(password, clientSalt, OPSLIMIT_INTERACTIVE, MEMLIMIT_INTERACTIVE)
```

4. Client sends (`username`, `clientHash`, `clientSalt`, `serverSalt`) to `POST /api/auth/signup/finish`.
5. Server computes

```
passwordHash = SHA-256(clientHash || ":" || serverSalt)
```


and stores (`username`, `clientSalt`, `serverSalt`, `passwordHash`).

Login flow:

1. Client sends username to `POST /api/auth/login/start`.
2. Server returns the stored `clientSalt` and `serverSalt`.
3. Client re-derives `clientHash` from the password and `clientSalt`.
4. Client sends (`username`, `clientHash`) to `POST /api/auth/login/finish`.
5. Server verifies: `SHA-256(clientHash || ":" || serverSalt) == stored passwordHash`.
6. On success, server generates a UUID session token, stores it in an in-memory map, and returns it.

3.4.3 Security properties

- **Password never transmitted:** Only the Argon2id-derived hash leaves the client, not the plaintext password.
- **Memory-hard key derivation:** Argon2id (the Argon2 variant resistant to both side-channel and GPU attacks) makes offline brute-force costly even with the `clientSalt`.
- **Two-salt architecture:** The `clientSalt` protects against rainbow tables on the client-derived hash; the `serverSalt` adds a second layer so that compromising one salt is insufficient.

- **Session tokens:** UUID-based bearer tokens stored in server memory; no JWT signature verification overhead, but tokens do not survive server restarts.

3.5 Real-time event system

3.5.1 WebSocket architecture

The server maintains a per-user socket registry: a `Map<userId, Set<WebSocket>` that tracks all active connections for each authenticated user. This design supports multiple simultaneous sessions (e.g., a user with both a desktop and mobile browser open).

Connection lifecycle:

1. Client opens WebSocket to `/ws?token=<bearer_token>`.
2. Server validates the token against the session map.
3. If valid, the socket is added to the user's socket set, and a `{"type": "hello", "authenticated": true}` message is sent.
4. If invalid, the server sends `{"type": "hello", "authenticated": false}` and closes the connection.
5. On socket close, the socket is removed from the user's set.

3.5.2 Event types and broadcast semantics

Six event types trigger WebSocket broadcasts:

- `entity.created`, `entity.updated`, `entity.deleted` — broadcast to all connected users.
- `trial.created`, `trial.updated` — broadcast to all connected users.
- `notification.created` — sent only to the specific recipient user's sockets.

Broadcast events are fire-and-forget: the server does not wait for client acknowledgment. If a client's socket is not in the `OPEN` state, the message is silently skipped. This design prioritizes throughput over guaranteed delivery, consistent with the system's eventual-consistency approach where clients can always re-fetch authoritative state via REST.

3.6 Data collection and population

Data sources are mixed:

- **Static map metadata:** Extracted from TypeScript source files (`mapPointToConstructName.ts`) containing room and lift names for all seven floors. Each floor defines a mapping from numeric point indices to human-readable construct names (e.g., point 0 → “O2.101”, point 55 → “A1.109”).
- **Automatic map entity provisioning:** At backend startup, the server iterates over all floors and construct names, generating deterministic entity IDs via `SHA-256(normalize("Floor N: RoomName")).slice(0, 24)`. For each location, an entity of type `map_location` is created (or verified to exist) with appropriate title, body, and tags (`["map", "room-or-stairs"]`). This produces approximately 100+ stable map entities that serve as collaborative thread anchors.
- **Synthetic interaction data:** Generated through UI interactions, API calls, and automated test scenarios. The API test suite creates users, posts, comments, references, follows, notifications, and trials to exercise the full entity lifecycle.

3.7 Indexing and query-planning strategy

To support growth in entity and interaction volume, the data model is designed around predictable access paths:

- **Entity listing/filtering:** Primary access patterns include filtering by `type` and `parentEntityId` (for comment retrieval under a parent entity). Index candidates include composite indexes on `(type, createdAt)` and `(parentEntityId, createdAt)` for efficient sorted scans.
- **Notification retrieval:** Inbox-like queries filter by `userId` and optionally by `read` status, then sort by `createdAt`. A composite index on `(userId, read, createdAt)` would support this pattern efficiently.
- **Edge traversals:** ArangoDB automatically indexes `_from` and `_to` attributes on edge collections, so traversal cost is primarily controlled by graph diameter and branching factor rather than collection size.
- **Map entity lookup:** The deterministic ID generation for map entities (`entity_map_<hash>`) enables $O(1)$ document retrieval by `_key` without requiring secondary indexes or fuzzy matching.
- **Full-text search:** The current substring-based search (`CONTAINS ( LOWER ( . . . ) )`) requires a full collection scan. For production workloads, this would be replaced by ArangoSearch views with inverted indexes for sub-linear lookup.

Although this project does not include a full benchmark matrix, these index assumptions were validated qualitatively during test scenarios by observing responsive API behavior for core reads and traversals.

3.8 Database queries and operations

3.8.1 Relational algebra formalization

Let E denote the `entities` collection and R the `entity_references` edge collection. Key operations can be expressed in relational algebra:

All comments under parent entity p :

$$\sigma_{\text{type}='comment' \wedge \text{parentEntityId}=p}(E)$$

Entities that reference target entity t (inbound reference lookup):

$$\pi_{E.*}(\sigma_{R._to='entities/'||t}(R) \bowtie_{R._from='entities/'||E.id} E)$$

Follow-based notification fan-out for entity e : Let F denote the `follows` collection.

$$\pi_{F.userId}(\sigma_{F.entityId=e}(F))$$

This produces the set of user IDs who should receive a notification when entity e is updated.

3.8.2 AQL query implementations

The following AQL queries are used in the backend, with parameterized bindings (`@param`) to prevent injection:

1. Entity listing with optional filters:

Listing 3.1: Entity listing query

```
1 FOR e IN entities
2   FILTER (@type == null OR e.type == @type)
3   FILTER (@parent == null OR e.parentEntityId == @parent)
```

```

4  SORT e.createdAt DESC
5  RETURN e

```

2. Reference edge synchronization (upsert pattern):

Listing 3.2: Reference edge synchronization

```

1  FOR targetKey IN @targets
2    LET targetDoc = DOCUMENT(CONCAT("entities/", targetKey))
3    FILTER targetDoc != null
4    INSERT {
5      _key: SHA256(CONCAT(@fromId, ":", targetKey)),
6      _from: @fromId,
7      _to: CONCAT("entities/", targetKey),
8      createdAt: @createdAt
9    } INTO entity_references OPTIONS { overwriteMode: "replace" }

```

3. Inbound reference discovery:

Listing 3.3: Inbound reference lookup

```

1  FOR edge IN entity_references
2    FILTER PARSE_IDENTIFIER(edge._to).key IN @ids
3    COLLECT from = edge._from
4    LET entity = DOCUMENT(from)
5    FILTER entity != null
6    RETURN entity

```

4. Shortest path between entities (graph traversal):

Listing 3.4: Degree-of-separation query

```

1  LET start = DOCUMENT(CONCAT("entities/", @from))
2  LET target = DOCUMENT(CONCAT("entities/", @to))
3  FILTER start != null AND target != null
4  FOR vertex IN ANY SHORTEST_PATH
5    start TO target entity_references
6  RETURN vertex

```

This query uses ArangoDB's native graph traversal engine with the ANY direction qualifier, treating reference edges as bidirectional for path discovery. The SHORTEST_PATH operation is implemented using a bidirectional BFS internally by ArangoDB, achieving $O(V + E)$ complexity on the reference subgraph.

5. Full-text search (substring matching):

Listing 3.5: Full-text search query

```

1  FOR e IN entities
2    FILTER @q == "" OR
3      CONTAINS(LOWER(CONCAT_SEPARATOR(" ",
4        e.id, e.title, e.body)), @q)
5    SORT e.createdAt DESC
6    RETURN e

```

6. Edge cleanup on entity deletion:

Listing 3.6: Reference edge cleanup

```

1  FOR edge IN entity_references

```



```
2  FILTER edge._from == @docId OR edge._to == @docId
3  REMOVE edge IN entity_references
```

This ensures referential integrity: when an entity is deleted, all inbound and outbound reference edges are removed, preventing dangling pointers in graph traversals.

3.9 Trial system design

3.9.1 Motivation and governance model

The trial system implements a structured community dispute-resolution mechanism inspired by formal adjudication processes. It is designed as a governance experiment: a sandbox where communities can deliberate, propose judges, vote, and reach recorded outcomes. The system is explicitly not a legal framework—it is a collaborative tool for transparent decision-making.

3.9.2 Trial state machine

Each trial follows a three-phase lifecycle:

Phase 1: Pending Agreement (**pending_agreement**).

- A plaintiff creates a trial by specifying a defendant username.
- Either party can propose a list of judges via `POST /api/trials/:id/propose-judges`.
- Turn-taking is enforced: the `lastProposedBy` field prevents the same party from proposing twice consecutively.
- The other party may counter-propose with a different judge list or accept the current proposal via the endpoint `POST /api/trials/:id/accept-judges`.
- All proposals and acceptances are recorded in the `judgeNegotiationHistory` array for auditability.
- When one party accepts the other's proposal, the trial transitions to `active` status.

Phase 2: Active (**active**).

- Only the agreed judges may cast votes via `POST /api/trials/:id/vote`.
- Vote options are: `"plaintiff"`, `"defendant"`, or `"no_winner"`.
- Discussion continues on the trial's linked entity thread, where comments are tagged with participant roles (plaintiff, defendant, judge, or spectator).

Phase 3: Resolved (**resolved**).

- When all agreed judges have voted, the outcome is calculated by plurality: the option with the most votes wins. Ties resolve to `"no_winner"`.
- The trial status changes to `resolved` and the outcome is permanently recorded.

3.9.3 Data integrity constraints

The trial system enforces several invariants through application-level validation:

- Only the plaintiff or defendant can propose or accept judges.
- A party cannot accept their own proposal.

- A party cannot propose twice in succession.
- Only users listed in `agreedJudges` can vote.
- Each judge can vote exactly once.
- The outcome is immutable once calculated.

3.10 Application architecture

3.10.1 System components

The system consists of four architectural layers:

- **Frontend (Vite + TypeScript):** A single-page application with five main views: Landing, Map View, Shortest Path, Traveling Salesman, and Collaborative Hub. The collaborative hub further subdivides into eight sub-pages: Hub, Authentication, Entities, Trials, Research, Notifications, Activity, and Tutorial.
- **Backend (Express 5):** A stateless HTTP/WebSocket server exposing 28 REST endpoints organized into six groups: authentication (5 endpoints), entities (5), entity interactions (2), notifications (2), trials (6), research (3), and system (2: health check + activity feed). Session state is maintained in an in-memory map.
- **Database (ArangoDB 3.11):** Provides durable storage, AQL query execution, and native graph traversal capabilities. Runs as a Docker container with persistent volume.
- **Realtime channel (WebSocket):** A lightweight event bus that broadcasts state-change notifications to authenticated clients, enabling live UI updates without polling.

3.10.2 API endpoint summary

Table 3.3 provides the complete API surface.

Table 3.3: REST API endpoint summary (28 endpoints).

Group	Endpoint	Purpose
Auth	POST <code>/api/auth/signup/{start,finish}</code>	Two-phase signup
Auth	POST <code>/api/auth/login/{start,finish}</code>	Two-phase login
Auth	GET <code>/api/auth/me</code>	Current user info
Entities	GET <code>/api/entities</code>	List (filter by type, parent)
Entities	GET <code>/api/map/entity</code>	Map entity by location
Entities	POST <code>/api/entities</code>	Create with references
Entities	PATCH <code>/api/entities/:id</code>	Update (creator only)
Entities	DELETE <code>/api/entities/:id</code>	Delete with edge cleanup
Social	POST <code>/api/entities/:id/follow</code>	Follow entity
Social	POST <code>/api/entities/:id/unfollow</code>	Unfollow entity
Notif.	GET <code>/api/notifications</code>	User inbox
Notif.	POST <code>/api/notifications/:id/read</code>	Mark read
Trials	GET, POST <code>/api/trials</code>	List/create trials
Trials	GET <code>/api/trials/:id</code>	Trial details
Trials	POST <code>.../{propose,accept,vote}</code>	Lifecycle actions
Research	GET <code>/api/research/references</code>	Inbound references

Group	Endpoint	Purpose
Research	GET /api/research/fulltext	Text search
Research	GET /api/research/degree	Shortest path
System	GET /api/health	Health check
System	GET /api/activity	Activity feed

3.10.3 Map data structure and rendering

The campus map covers seven floors, each represented as:

- An HTML/CSS grid layout generated via Hyperscript, with clickable room elements carrying `data-construct-name` and `data-isconstruct` attributes for programmatic interaction.
- A graph structure defined as an edge list (`[number, number][ ]`), where each number is a point index corresponding to a physical location.
- A coordinate array (`points[index] = [x, y]`) mapping point indices to pixel positions for path visualization.
- Rendering primitives (`mapPrimitives.ts`) for rooms, lifts, stairs, classrooms, and libraries.

Four lift positions (A1-left, A1-right, A2-left, A2-right) are shared across floors, serving as inter-floor transfer points for multi-floor routing. Path visualization uses CSS-based line drawing with Anime.js animations for smooth path tracing.

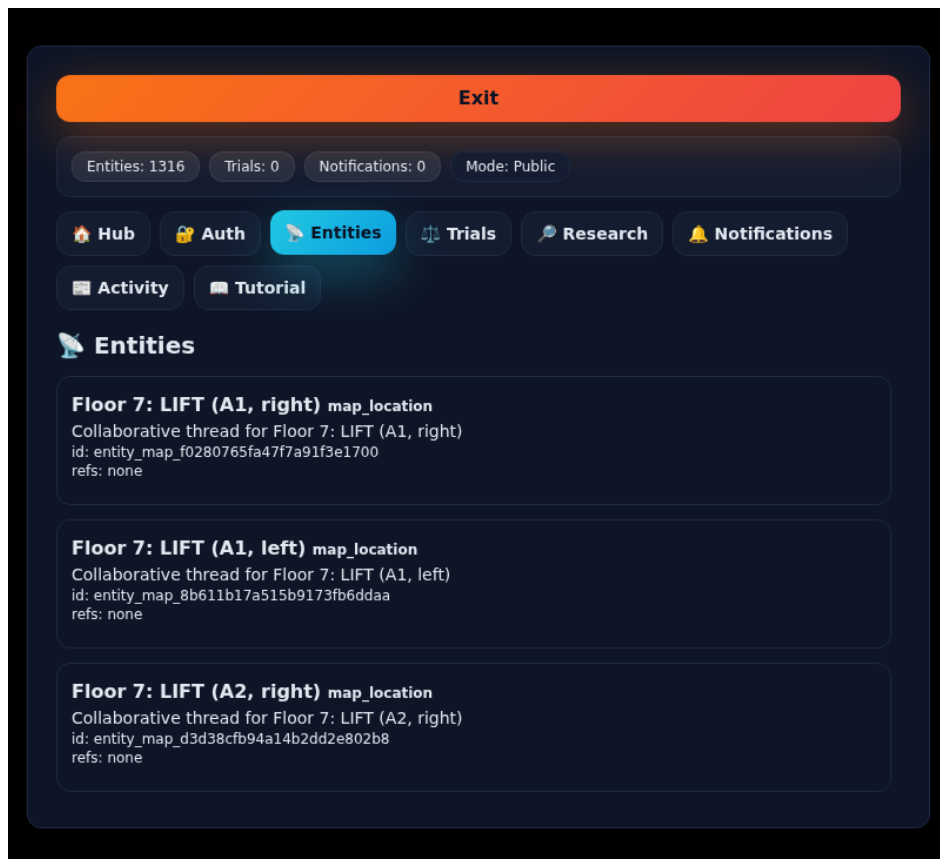


Figure 3.1: Collaborative page UI showing entity creation, discussion threads, and research panels.

3.11 User interface design

The collaborative UI is organized into eight focused sub-pages, each addressing a specific task domain:

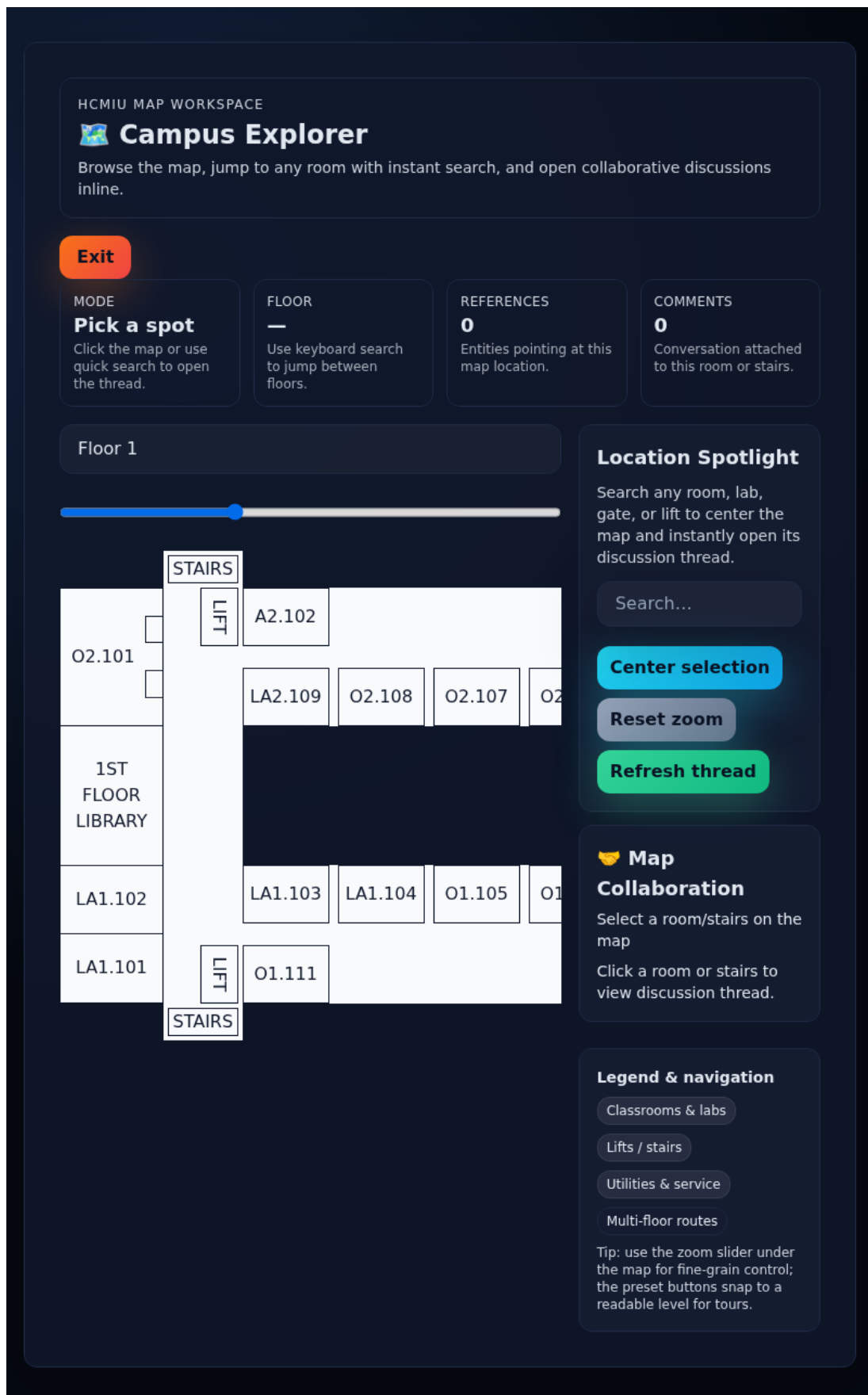


Figure 3.2: Map view page with floor browsing and room-to-entity deep linking.

1. **Hub:** Overview dashboard with entity and trial statistics.
2. **Authentication:** Signup/login forms with client-side libsodium key derivation.
3. **Entities:** Entity creation with reference selection (search, map picker, or manual ID entry), comment threads, and follow/unfollow controls.
4. **Trials:** Trial creation, interactive judge negotiation interface, voting panel, and trial-linked discussion with participant role tags.
5. **Research:** Three research modes—inbound reference lookup, full-text search, and degree-of-separation path finder.
6. **Notifications:** Real-time notification inbox with read/unread status.
7. **Activity:** Chronological feed of the 50 most recent system-wide events.
8. **Tutorial:** Interactive user guide with feature walkthroughs.

This segmentation reduces interaction complexity, supports task-oriented workflows, and enables independent development and testing of each sub-page.

Chapter 4

Discussion & Implementation

4.1 Implementation process

The implementation followed an incremental, test-driven path:

1. **Infrastructure:** Establish Docker Compose configuration with ArangoDB container, health checks, and volume persistence.
2. **Database initialization:** Implement collection creation logic with existence checks and edge collection type specification (`type: 3` for edge collections).
3. **Map provisioning:** Auto-generate map entities for all seven floors at startup using deterministic SHA-256-based IDs.
4. **Authentication:** Implement the two-phase SRP-inspired signup/login protocol with libsodium integration.
5. **Entity CRUD:** Add entity creation with reference array synchronization to the edge collection, update with re-synchronization, and delete with edge cleanup.
6. **Social features:** Layer follow/unfollow mechanics, notification generation on entity updates for followers, and the activity feed.
7. **Research APIs:** Implement inbound reference lookup, full-text search, and degree-of-separation graph traversal.
8. **Trial system:** Build the three-phase trial lifecycle with judge negotiation, voting, and outcome calculation.
9. **Frontend integration:** Connect all collaborative UI pages, implement WebSocket real-time refresh, and add map-to-entity deep linking.
10. **Testing:** Develop comprehensive API test suite and five Puppeteer-based end-to-end demo scenarios.

4.2 Testing methodology

4.2.1 API test suite

The API test suite (`api.test.mjs`, 243 lines) uses Node.js's native test runner to exercise the complete backend workflow:

- **Authentication:** Signup with libsodium client-side hashing, login verification, and session token management.
- **Entity lifecycle:** Create, read, update, and delete operations with validation of reference edge synchronization.

- **Social interactions:** Follow/unfollow entities, verify notification creation on new comments, and check notification read marking.
- **Trial workflow:** Create trial, propose judges, counter-propose, accept, vote, and verify outcome calculation.
- **Research queries:** Inbound reference lookup, full-text search, and degree-of-separation path traversal.
- **Activity feed:** Verify chronological ordering and content correctness.

4.2.2 End-to-end test suite

The E2E suite (`e2e.test.mjs`, 295 lines) runs against the full Docker Compose stack to validate production-like behavior, exercising frontend–backend–database integration paths.

4.2.3 Demo video scenarios

Five Puppeteer-based demo scenarios serve as both automated acceptance tests and documentation artifacts:

1. **Comprehensive demo** (695 lines): Full feature walkthrough producing an MP4 video.
2. **Multi-user real-time** (358 lines): Two-user collaboration with live WebSocket notifications.
3. **Trial system** (321 lines): Complete trial lifecycle from creation through resolution.
4. **Map navigation** (400 lines): Shortest path and traveling-salesman route planning.
5. **Research knowledge** (328 lines): Reference lookup, full-text search, and degree-of-separation queries.

These scenarios use Puppeteer 24’s screencast API to record WebM video, which is then converted to MP4 with `ffmpeg` for distribution.

4.3 Results and analysis

4.3.1 Functional outcomes

- **Map entities:** Over 100 map entities are auto-generated across seven floors, each serving as a stable collaborative thread anchor. The deterministic ID scheme ensures idempotent provisioning across server restarts.
- **Reference graph:** Inbound reference lookup, full-text search, and shortest-path queries operate correctly on the `entity_references` edge collection. The `ANY SHORTEST_PATH` traversal provides bidirectional path discovery.
- **Trial workflow:** The three-phase trial lifecycle (negotiation, voting, resolution) functions correctly with all invariant checks (turn-taking, judge-only voting, immutable outcomes).
- **Real-time updates:** WebSocket broadcasts trigger immediate UI refresh across all connected clients, verified in the multi-user demo scenario.
- **Research endpoints:** The three research APIs (references, full-text, degree) are reusable by external scripts and future dashboards.

4.3.2 Database-level observations

- **Unified query language:** AQL’s ability to combine document operations (`FOR ... FILTER ... RETURN`) with graph traversals (`SHORTEST_PATH`, `TRAVERSE`) eliminates the need for application-level data stitching that would be required in a polyglot architecture.
- **Edge synchronization:** The upsert pattern for reference edges (using SHA-256 deterministic keys and `overwrite-Mode: "replace"`) ensures consistency between the entity’s `references[]` array and the edge collection without requiring explicit transactions.
- **Graph vs. relational expressiveness:** The degree-of-separation query is a single AQL statement (`ANY SHORTEST_PATH`), compared with a recursive Common Table Expression (CTE) in SQL that would require explicit depth tracking, cycle detection, and termination conditions.
- **Schema flexibility:** Adding the `judgeNegotiationHistory` array to trials and the `tags[]` array to entities required no schema migration—documents were simply written with the new fields, and existing documents continued to function with implicit `null` values.
- **Trade-off awareness:** The generic entity model accelerated feature delivery but required disciplined type checks at the application level to avoid semantic drift (e.g., treating a `map_location` entity as a regular post).

4.3.3 Algorithmic component analysis

Shortest path correctness. BFS guarantees optimal shortest paths for unweighted graphs. The inter-floor routing extension exhaustively evaluates all four lift positions, ensuring optimality across the combined source-floor and destination-floor path lengths.

TSP optimality. The Held–Karp bitmask DP implementation guarantees optimal solutions. For the practical limit of $n \leq 15$ destinations, the $O(2^n \cdot n^2)$ complexity yields a manageable number of operations, and the implementation uses a flat `Uint32Array` for the DP table, avoiding object-allocation overhead and enabling cache-friendly memory access patterns.

Graph traversal in AQL. ArangoDB’s `SHORTEST_PATH` operation uses bidirectional BFS internally, which has optimal worst-case complexity for unweighted edge collections. The `ANY` direction qualifier allows the traversal to follow edges in both directions, which is semantically appropriate for reference relationships (“A references B” implies a conceptual connection in both directions).

4.4 Challenges and solutions

- **Challenge:** Balancing a generic entity schema with feature-specific behavior. Entities serve as posts, comments, map locations, and trial discussion threads, each requiring different validation and display logic.
Solution: Use a `type` discriminator field combined with dedicated collections for complex workflow state (e.g., the `trials` collection for dispute metadata, linked to a companion entity for discussion).
- **Challenge:** Maintaining real-time consistency between REST writes and UI state across multiple concurrent clients.
Solution: Broadcast `WebSocket` events on every state mutation, and have clients re-fetch canonical state from REST endpoints upon event receipt. This full-refresh approach trades bandwidth for simplicity and eliminates state-divergence bugs.
- **Challenge:** Establishing stable, predictable linkage between map coordinates and collaborative threads across server restarts.
Solution: Deterministic entity ID generation using `SHA-256(normalize(floorName))` ensures that the same room always maps to the same entity ID, enabling idempotent provisioning.

- **Challenge:** Ensuring referential integrity when entities are deleted (dangling edges in the reference graph).
Solution: On entity deletion, explicitly remove all inbound and outbound edges from `entity_references` before removing the entity document.
- **Challenge:** Implementing secure password handling without transmitting plaintext passwords.
Solution: The SRP-inspired two-phase protocol with client-side Argon2id key derivation ensures the server only receives a derived hash, reducing exposure in transit and at rest.

4.5 Threats to validity

- **External validity:** The dataset and interaction patterns are campus-specific. Behavior on larger institutions with significantly denser interaction graphs, more concurrent users, and more diverse entity types may differ from observed results.
- **Performance validity:** No formal throughput or latency benchmarks were conducted. Performance observations (responsive API behavior, sub-second query times) are qualitative and based on developer-scale workloads. Scalability conclusions require systematic load testing with synthetic workloads.
- **Behavioral validity:** User interactions are synthetic (developer-driven and automated-test-generated) rather than from a deployed community. Real-world usage patterns—moderation dynamics, reference network density, trial participation rates—may differ substantially.
- **Security validity:** The authentication protocol has not undergone formal cryptographic audit. While the design follows established patterns (Argon2id, two-salt architecture), implementation subtleties could introduce vulnerabilities not captured by functional tests.
- **Construct validity:** The comparison between multi-model and polyglot approaches (Table 3.1) is based on architectural analysis rather than a controlled experiment with both implementations built and measured.

Chapter 5

Conclusion & Future Work

5.1 Summary of key findings

This project demonstrates that ArangoDB is well-suited for a collaborative campus map application where content flexibility and relationship-centric queries are equally important. Key findings include:

1. **Multi-model fit:** Document storage supports flexible, evolving user-generated content (entities with varying types, nested arrays, and optional fields), while graph edges and native AQL traversals enable deep relationship queries (reference discovery, degree of separation, notification fan-out) without external graph infrastructure.
2. **Developer productivity:** A single query language (AQL) and single deployment target (ArangoDB) reduced context-switching overhead during iterative feature development, compared with maintaining two separate query languages and deployment pipelines in a polyglot architecture.
3. **Algorithmic integration:** Classical graph algorithms (BFS for shortest path, Held–Karp for TSP) integrate naturally with the graph-modeled campus data, providing provably optimal solutions within practical computational bounds.
4. **Governance experiment:** The trial system demonstrates that structured dispute-resolution workflows can be modeled as database state machines with full auditability, though real-world governance effectiveness remains unvalidated.

5.2 Reflection on objectives

The primary objectives were achieved:

- **Objective 1 (Wayfinding):** The system provides interactive floor browsing, single-source shortest-path routing, multi-floor routing via lifts, and optimal multi-stop planning, reducing cognitive load for campus navigation.
- **Objective 2 (Advanced DBMS):** The project exercises ArangoDB’s document storage, graph traversal, edge collection management, and AQL query capabilities across 28 API endpoints.
- **Objective 3 (Collaborative features):** Entity references, follows, notifications, full-text search, degree-of-separation queries, and the trial system provide a comprehensive relationship-first collaborative platform.
- **Objective 4 (Security):** The SRP-inspired authentication protocol with client-side Argon2id key derivation protects user credentials from plaintext exposure.
- **Objective 5 (Validation):** The API test suite, E2E test suite, and five Puppeteer demo scenarios provide automated verification of all major workflows.

5.3 Advanced database implications

From an advanced DBMS perspective, this work contributes several observations:

- **Multi-model advantage:** For applications with mixed document-and-graph workloads, a multi-model DBMS eliminates the impedance mismatch and operational overhead of polyglot persistence, at the cost of potentially sub-optimal performance for any single workload pattern compared with a specialized engine.
- **Edge collection patterns:** The deterministic-key upsert pattern for reference edges provides a practical approach to maintaining consistency between denormalized document arrays and normalized edge collections without explicit transactions.
- **Graph query expressiveness:** AQL's native `SHORTEST_PATH` and traversal operators enable concise expression of relationship queries that would require complex recursive CTEs in SQL, reducing both code complexity and maintenance burden.
- **Schema evolution:** Document databases' schema flexibility is particularly valuable during rapid prototyping, but requires application-level type discipline to prevent semantic drift in polymorphic collections.

5.4 Future work

- **ArangoSearch integration:** Replace substring-based full-text search with ArangoSearch views using inverted indexes and BM25-based relevance ranking, enabling sub-linear search complexity and ranked retrieval quality comparable to dedicated search engines.
- **Fine-grained authorization:** Introduce role-based access control (trusted editors, moderators, administrators) with moderation policies (temporary suspensions, content flagging, audit trails) to support safe community governance at scale.
- **Performance benchmarking:** Design and execute systematic load tests with synthetic workloads (varying entity counts, reference density, concurrent users) to produce quantitative latency percentile distributions and throughput ceilings, enabling evidence-based capacity planning.
- **Indoor positioning integration:** Incorporate optional positioning feeds (BLE beacons, Wi-Fi fingerprinting) and accessibility-aware routing constraints (elevator-only paths, wheelchair-accessible routes, restricted-zone avoidance) to bridge the gap between graph-based and sensor-based navigation.
- **Conflict-resolution analytics:** Formalize quantitative metrics from trial data—time-to-resolution distributions, participation balance (judge response rates), appeal patterns, outcome fairness measures—to enable data-driven governance design improvements.
- **Offline support:** Implement service worker caching and optimistic offline entity creation with sync-on-reconnect, enabling campus navigation in areas with poor connectivity.
- **Formal security audit:** Subject the authentication protocol and API authorization logic to formal cryptographic review and penetration testing to validate the security properties claimed in this report.

Bibliography

- [1] Renzo Angles and Claudio Gutierrez. The property graph database model. *Proceedings of the 12th International Conference on Extending Database Technology*, pages 236–247, 2008.
- [2] ArangoDB GmbH. Arangodb query language (aql) documentation. <https://docs.arangodb.com/stable/aql/>, 2025. Accessed: 2026-03-10.
- [3] ArangoDB GmbH. Multi-model database: Graph, document, and search. <https://arangodb.com/multi-model/>, 2025. Accessed: 2026-03-10.
- [4] Richard Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61–63, 1962.
- [5] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [6] Ian Fette and Alexey Melnikov. The websocket protocol. RFC 6455, 2011.
- [7] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [8] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. O’Reilly Media, 2nd edition, 2015.
- [9] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012.
- [10] Michael Stonebraker and Ugur Cetintemel. One size fits all? part 2: Benchmarking results. *CIDR*, 2005.
- [11] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.
- [12] Craig Zimring and Ruth C. Dalton. Wayfinding in complex buildings: An environmental and behavioral perspective. *Environment and Behavior*, 38(4):558–575, 2006.